Algoritmos de búsqueda

Notas de clase

Estructuras de datos y algoritmos

Facultad TIC - UPB

Winter 2026

Jorge Mario Londoño Peláez & Varias AI

February 10, 2026

Descripción de la unidad

Estudio de algoritmos de búsqueda lineal y binaria.

Contents

1 Definición del problema de la búsqueda **2**

1.1 La tabla de símbolos 2

1.1.1 API de la tabla de símbolos no ordenada 2

1.1.2 API de la tabla de símbolos ordenada 3

1.1.3 Convenciones sobre llaves y valores 3

2 La búsqueda secuencial **3**

2.1 Tabla de símbolos por lista simple no ordenada 3

2.1.1 Estructura del nodo 4

2.1.2 Implementación de las operaciones 4

2.1.3 Eficiencia de las operaciones 5

3 La búsqueda binaria **5**

3.1 Representación de la tabla de símbolos por medio de arreglo ordenado 5

3.2 Implementación de las operaciones de la tabla de símbolos 6

3.2.1 Operación rank(llave) 6

3.2.2 Operación get(llave) 6

3.2.3 Operación put(llave, valor) 6

3.2.4 Operación delete(llave) 7

3.3 Eficiencia de la búsqueda binaria 7

3.4 Comparación entre búsqueda secuencial y búsqueda binaria 8

1 Definición del problema de la búsqueda

El problema de la búsqueda es fundamental en la informática y se presenta en una variedad de aplicaciones. En su esencia, el problema consiste en encontrar un elemento específico dentro de un conjunto de datos. Este conjunto de datos puede ser una lista, un arreglo, una base de datos, o incluso la web. Formalmente, dado un conjunto de elementos y una llave de búsqueda, el objetivo es determinar si existe un elemento en el conjunto que coincida con la llave, y si existe, recuperar información asociada a ese elemento.

Algunos ejemplos de aplicaciones del problema de búsqueda son:

- Buscar un contacto en la aplicación de contactos del celular.
- Buscar un contenido en la web.
- Encontrar un estudiante por su `ID`.
- Buscar registros en una base de datos.

1.1 La tabla de símbolos

Una **tabla de símbolos** (también conocida como diccionario o mapa) es un Tipo de Dato Abstracto (TDA) que modela el problema de la búsqueda de manera eficiente. Una tabla de símbolos almacena un conjunto de pares llave-valor, donde cada llave es única y está asociada a un valor específico. Las tablas de símbolos permiten realizar operaciones de búsqueda, inserción y eliminación de elementos basándose en su llave.

Existen dos tipos principales de tablas de símbolos:

- **Tablas de símbolos no ordenadas:** En este tipo de tabla, las llaves no tienen un orden específico. La inserción de nuevos elementos se realiza de manera sencilla, pero la búsqueda puede requerir una exploración exhaustiva de todos los elementos.
- **Tablas de símbolos ordenadas:** En este tipo de tabla, las llaves se mantienen ordenadas según lo defina la comparación de llaves. Esto permite realizar búsquedas más eficientes, como la búsqueda binaria, pero la inserción y eliminación de elementos pueden ser más costosas debido a la necesidad de mantener el orden.

1.1.1 API de la tabla de símbolos no ordenada

La Application Programming Interface (API) básica de una tabla de símbolos no ordenada incluye las siguientes operaciones:

- `put(llave, valor)`: Inserta un nuevo par llave-valor en la tabla.
- `get(llave)`: Busca la llave en la tabla y devuelve el valor asociado. Si la llave no existe, devuelve un valor nulo o un valor por defecto.
- `delete(llave)`: Elimina el par llave-valor asociado a la llave dada.
- `contains(llave)`: Verifica si la llave existe en la tabla.
- `isEmpty()`: Verifica si la tabla está vacía.
- `size()`: Devuelve el número de pares llave-valor en la tabla.
- `keys()`: Devuelve un iterador sobre las llaves de la tabla.

1.1.2 API de la tabla de símbolos ordenada

Además de las operaciones de la tabla de símbolos no ordenada, la API de una tabla de símbolos ordenada incluye las siguientes operaciones:

- `min()`: Devuelve la llave mínima en la tabla.
- `max()`: Devuelve la llave máxima en la tabla.
- `floor(llave)`: Devuelve la llave más grande en la tabla que es menor o igual a la llave dada.
- `ceiling(llave)`: Devuelve la llave más pequeña en la tabla que es mayor o igual a la llave dada.
- `rank(llave)`: Devuelve el número de llaves en la tabla que son menores que la llave dada.
- `select(k)`: Devuelve la llave de rango k en la tabla (es decir, la llave que tiene k llaves menores que ella).
- `size(lo,hi)`: Devuelve el número de pares llave-valor cuya llave se encuentra entre lo y hi inclusive.
- `keys(lo,hi)`: Devuelve un iterador sobre las llaves de la tabla que se encuentran entre lo y hi inclusive.

1.1.3 Convenciones sobre llaves y valores

Para las implementaciones de tablas de símbolos en este capítulo, se adoptarán las siguientes convenciones:

- **Llaves no nulas:** No se permiten llaves con valor nulo.
- **Llaves únicas:** No pueden existir llaves duplicadas en la tabla. Cada llave es única. Si se intenta insertar una llave que ya existe, el nuevo valor reemplaza al valor anterior.
- **Mecanismo de eliminación:** El método `delete(llave)` es el mecanismo principal para eliminar un par llave-valor de la tabla. Algunas implementaciones también permiten la convención alternativa de usar `put(llave, null)` para eliminar, pero en nuestras implementaciones usaremos explícitamente `delete`.

En las siguientes secciones estudiaremos dos implementaciones fundamentales de tablas de símbolos: la búsqueda secuencial usando listas enlazadas (Sección 2) y la búsqueda binaria usando arreglos ordenados (Sección 3). Cada implementación tiene sus ventajas y desventajas en términos de eficiencia temporal y espacial.

2 La búsqueda secuencial

2.1 Tabla de símbolos por lista simple no ordenada

Una forma sencilla de implementar una tabla de símbolos es utilizando una lista enlazada simple no ordenada. En esta representación, cada elemento de la lista contiene un par llave-valor.

2.1.1 Estructura del nodo

Cada nodo de la lista enlazada contiene los siguientes campos:

- `llave`: La llave del par llave-valor.
- `valor`: El valor asociado a la llave.
- `siguiente`: Un puntero al siguiente nodo en la lista.

La tabla de símbolos también mantiene un contador `N` que representa el número total de pares llave-valor almacenados, lo cual permite implementar la operación `size()` en tiempo constante $O(1)$.

2.1.2 Implementación de las operaciones

A continuación, se describe la implementación de las operaciones principales de la tabla de símbolos utilizando pseudocódigo:

put(llave, valor)

```
procedure PUT(llave, valor)
    actual ← cabeza
    while actual ≠ null do
        if actual.llave = llave then
            actual.valor ← valor                ▷ Actualizar el valor si la llave ya existe
            return
        end if
        actual ← actual.siguiente
    end while
    nuevo ← Nodo(llave, valor)                  ▷ Crear un nuevo nodo
    nuevo.siguiente ← cabeza                    ▷ Apuntar el nuevo nodo al inicio de la lista
    cabeza ← nuevo                              ▷ Actualizar la cabeza de la lista
    N ← N + 1                                   ▷ Incrementar el contador de elementos
end procedure
```

get(llave)

```
procedure GET(llave)
    actual ← cabeza
    while actual ≠ null do
        if actual.llave = llave then
            return actual.valor                 ▷ Devolver el valor asociado a la llave
        end if
        actual ← actual.siguiente
    end while
    return null                                 ▷ Devolver null si la llave no se encuentra
end procedure
```

delete(llave)

```
procedure DELETE(llave)
    actual ← cabeza
    anterior ← null
    while actual ≠ null do
        if actual.llave = llave then
            if anterior = null then
                cabeza ← actual.siguiente                    ▷ Eliminar el nodo de la cabeza
            else
                anterior.siguiente ← actual.siguiente        ▷ Eliminar el nodo del medio o final
            end if
            N ← N − 1                                        ▷ Decrementar el contador de elementos
            return
        end if
        anterior ← actual
        actual ← actual.siguiente
    end while
end procedure
```

2.1.3 Eficiencia de las operaciones

A continuación, se presenta el orden de crecimiento del tiempo de ejecución de las operaciones principales de la tabla de símbolos implementada con una lista enlazada simple no ordenada:

- `put(llave, valor)`: $O(n)$ - Tiempo lineal, ya que en el peor de los casos se debe recorrer toda la lista para verificar si la llave ya existe antes de insertar.
- `get(llave)`: $O(n)$ - Tiempo lineal, ya que en el peor de los casos se debe recorrer toda la lista para encontrar la llave.
- `delete(llave)`: $O(n)$ - Tiempo lineal, ya que en el peor de los casos se debe recorrer toda la lista para encontrar la llave a eliminar.

Conclusión: La implementación de la tabla de símbolos con una lista simple no ordenada es sencilla, pero ineficiente para grandes conjuntos de datos debido a que las operaciones de búsqueda y eliminación tienen un tiempo de ejecución lineal.

3 La búsqueda binaria

3.1 Representación de la tabla de símbolos por medio de arreglo ordenado

Una tabla de símbolos se puede implementar utilizando dos arreglos paralelos: uno para las llaves y otro para los valores. Es requisito fundamental que el arreglo de llaves esté ordenado de forma ascendente. Esta organización permite aprovechar el algoritmo de búsqueda binaria para encontrar rápidamente la llave deseada.

- **Arreglo de llaves:** Almacena las llaves de la tabla de símbolos en orden ascendente.
- **Arreglo de valores:** Almacena los valores correspondientes a cada llave en la misma posición que su llave en el arreglo de llaves.

Es importante destacar que las llaves deben ser comparables, es decir, debe existir una forma de determinar si una llave es menor, igual o mayor que otra. Esto es esencial para mantener el orden en el arreglo de llaves y para realizar la búsqueda binaria de manera eficiente.

3.2 Implementación de las operaciones de la tabla de símbolos

3.2.1 Operación rank(llave)

La operación `rank(llave)` es fundamental en la búsqueda binaria. Devuelve el número de llaves en la tabla que son menores que la llave dada. En otras palabras, indica la posición que la llave dada debería ocupar en el arreglo de llaves si estuviera presente.

```
procedure RANK(llave)
    lo ← 0
    hi ← N − 1                                  ▷ N es el número de llaves en la tabla
    while lo ≤ hi do
        mid ← lo + ⌊(hi − lo)/2⌋                ▷ La división debe ser entera
        if llave < llaves[mid] then
            hi ← mid − 1
        else if llave > llaves[mid] then
            lo ← mid + 1
        else
            return mid                          ▷ Se encontró la llave
        end if
    end while
    return lo          ▷ No se encontró la llave, devolver la posición donde debería estar
end procedure
```

3.2.2 Operación get(llave)

La operación `get(llave)` busca la llave en el arreglo de llaves utilizando la búsqueda binaria. Si la llave se encuentra, devuelve el valor asociado en el arreglo de valores. Si la llave no se encuentra, devuelve nulo.

```
procedure GET(llave)
    i ← rank(llave)
    if i < N and llaves[i] = llave then
        return valores[i]
    else
        return nulo                             ▷ La llave no se encontró
    end if
end procedure
```

3.2.3 Operación put(llave, valor)

La operación `put(llave, valor)` inserta un nuevo par llave-valor en la tabla de símbolos. Si la llave ya existe, actualiza el valor asociado. Si la llave no existe, inserta la llave y el valor en la posición correcta para mantener el orden del arreglo de llaves.

Nota: El siguiente pseudocódigo asume que los arreglos tienen capacidad suficiente. En una implementación real, si N alcanza la capacidad del arreglo, sería necesario redimensionarlo (típicamente duplicando su tamaño) antes de insertar.

```
procedure PUT(llave, valor)
    i ← rank(llave)
    if i < N and llaves[i] = llave then
        valores[i] ← valor                    ▷ Actualizar el valor si la llave ya existe
        return
    end if
                                              ▷ Desplazar llaves y valores para hacer espacio
    for j ← N down to i + 1 do
        llaves[j] ← llaves[j − 1]
        valores[j] ← valores[j − 1]
    end for
    llaves[i] ← llave
    valores[i] ← valor
    N ← N + 1
end procedure
```

3.2.4 Operación delete(llave)

La operación `delete(llave)` elimina la llave y su valor asociado de la tabla de símbolos.

```
procedure DELETE(llave)
    i ← rank(llave)
    if i < N and llaves[i] = llave then     ▷ La llave fue encontrada, proceder a eliminarla
        for j ← i to N − 2 do
            llaves[j] ← llaves[j + 1]
            valores[j] ← valores[j + 1]
        end for
        N ← N − 1
        llaves[N] ← null                    ▷ Limpiar referencia para facilitar recolección de basura
        valores[N] ← null
    end if
end procedure
```

3.3 Eficiencia de la búsqueda binaria

A continuación, se presenta el orden de crecimiento del tiempo de ejecución de las operaciones principales de la tabla de símbolos implementada con arreglos ordenados y búsqueda binaria:

- `rank(llave)`: $O(\lg n)$ - Tiempo logarítmico, debido a la búsqueda binaria.
- `get(llave)`: $O(\lg n)$ - Tiempo logarítmico, debido a la búsqueda binaria.
- `put(llave, valor)`: $O(n)$ - Tiempo lineal, ya que en el peor de los casos se deben mover todos los elementos del arreglo para insertar la nueva llave.
- `delete(llave)`: $O(n)$ - Tiempo lineal, ya que en el peor de los casos se deben mover todos los elementos del arreglo para eliminar la llave.

Conclusión: La implementación de la tabla de símbolos con arreglos ordenados y búsqueda binaria ofrece una mejora significativa en el tiempo de búsqueda en comparación con la búsqueda secuencial. Sin embargo, las operaciones de inserción y eliminación siguen siendo costosas debido a la necesidad de mantener el orden del arreglo.

3.4 Comparación entre búsqueda secuencial y búsqueda binaria

La siguiente tabla compara el orden de crecimiento del tiempo de ejecución de las operaciones principales en ambas implementaciones:

Operación	**Búsqueda secuencial (lista enlazada)**	**Búsqueda binaria (arreglo ordenado)**
`get(llave)`	$O(n)$	$O(\lg n)$
`put(llave, valor)`	$O(n)$	$O(n)$
`delete(llave)`	$O(n)$	$O(n)$

Observaciones:

- La búsqueda binaria es significativamente más rápida para operaciones de consulta (`get`), con complejidad logarítmica frente a la lineal de la búsqueda secuencial.
- Ambas implementaciones tienen complejidad lineal para las operaciones de inserción y eliminación, aunque por razones diferentes: la búsqueda secuencial debe recorrer la lista, mientras que la búsqueda binaria debe desplazar elementos para mantener el orden.
- La búsqueda binaria es preferible cuando las operaciones de consulta son más frecuentes que las de modificación. Para aplicaciones con muchas inserciones y eliminaciones, estructuras más avanzadas como los árboles balanceados ofrecen mejor desempeño.